

## **Chương 2: KỸ THUẬT TÌM KIẾM (SEARCHING)**

### **2.1. Khái quát về tìm kiếm**

Trong thực tế, khi thao tác, khai thác dữ liệu chúng ta hầu như lúc nào cũng phải thực hiện thao tác tìm kiếm. Việc tìm kiếm nhanh hay chậm tùy thuộc vào trạng thái và trật tự của dữ liệu trên đó. Kết quả của việc tìm kiếm có thể là không có (không tìm thấy) hoặc có (tìm thấy). Nếu kết quả tìm kiếm là có tìm thấy thì nhiều khi chúng ta còn phải xác định xem vị trí của phần tử dữ liệu tìm thấy là ở đâu? Trong phạm vi của chương này chúng ta tìm cách giải quyết các câu hỏi này.

Trước khi đi vào nghiên cứu chi tiết, chúng ta giả sử rằng mỗi phần tử dữ liệu được xem xét có một thành phần khóa (Key) để nhận diện, có kiểu dữ liệu là T nào đó, các thành phần còn lại là thông tin (Info) liên quan đến phần tử dữ liệu đó. Như vậy mỗi phần tử dữ liệu có cấu trúc dữ liệu như sau:

```
typedef struct DataElement
{
    T      Key;
    InfoType Info;
} DataType;
```

Trong tài liệu này, khi nói tới giá trị của một phần tử dữ liệu chúng ta muốn nói tới giá trị khóa (Key) của phần tử dữ liệu đó. Để đơn giản, chúng ta giả sử rằng mỗi phần tử dữ liệu chỉ là thành phần khóa nhận diện.

Việc tìm kiếm một phần tử có thể diễn ra trên một dãy/mảng (tìm kiếm nội) hoặc diễn ra trên một tập tin/ file (tìm kiếm ngoại). Phần tử cần tìm là phần tử cần thỏa mãn điều kiện tìm kiếm (thường có giá trị bằng giá trị tìm kiếm). Tùy thuộc vào từng bài toán cụ thể mà điều kiện tìm kiếm có thể khác nhau song chung quy việc tìm kiếm dữ liệu thường được vận dụng theo các thuật toán trình bày sau đây.

### **2.2. Các giải thuật tìm kiếm nội (Tìm kiếm trên dãy/mảng)**

#### **2.2.1. Đặt vấn đề**

Giả sử chúng ta có một mảng M gồm N phần tử. Vấn đề đặt ra là có hay không phần tử có giá trị bằng X trong mảng M? Nếu có thì phần tử có giá trị bằng X là phần tử thứ mấy trong mảng M?

#### **2.2.2. Tìm tuyến tính (Linear Search)**

Thuật toán tìm tuyến tính còn được gọi là Thuật toán tìm kiếm tuần tự (Sequential Search).

##### **a. Tư tưởng:**

Lần lượt so sánh các phần tử của mảng M với giá trị X bắt đầu từ phần tử đầu tiên cho đến khi tìm đến được phần tử có giá trị X hoặc đã duyệt qua hết tất cả các phần tử của mảng M thì kết thúc.

**b. Thuật toán:**

```
B1: k = 1 //Duyệt từ đầu mảng
B2: IF M[k] ≠ X AND k ≤ N //Nếu chưa tìm thấy và cũng chưa duyệt hết mảng
    B2.1: k++
    B2.2: Lặp lại B2
B3: IF k ≤ N
    Tìm thấy tại vị trí k
B4: ELSE
    Không tìm thấy phần tử có giá trị X
B5: Kết thúc
```

**c. Cài đặt thuật toán:**

Hàm LinearSearch có prototype:

```
int LinearSearch (T M[], int N, T X);
```

Hàm thực hiện việc tìm kiếm phần tử có giá trị X trên mảng M có N phần tử. Nếu tìm thấy, hàm trả về một số nguyên có giá trị từ 0 đến N-1 là vị trí tương ứng của phần tử tìm thấy. Trong trường hợp ngược lại, hàm trả về giá trị -1 (không tìm thấy). Nội dung của hàm như sau:

```
int LinearSearch (T M[], int N, T X)
{
    int k = 0;
    while (M[k] != X && k < N)
        k++;
    if (k < N)
        return (k);
    return (-1);
}
```

**d. Phân tích thuật toán:**

- Trường hợp tốt nhất khi phần tử đầu tiên của mảng có giá trị bằng X:

Số phép gán:  $G_{min} = 1$

Số phép so sánh:  $S_{min} = 2 + 1 = 3$

- Trường hợp xấu nhất khi không tìm thấy phần tử nào có giá trị bằng X:

Số phép gán:  $G_{max} = 1$

Số phép so sánh:  $S_{max} = 2N + 1$

- Trung bình:

Số phép gán:  $G_{avg} = 1$

Số phép so sánh:  $S_{avg} = (3 + 2N + 1) : 2 = N + 2$

**e. Cải tiến thuật toán:**

Trong thuật toán trên, ở mỗi bước lặp chúng ta cần phải thực hiện 2 phép so sánh để kiểm tra sự tìm thấy và kiểm soát sự hết mảng trong quá trình duyệt mảng. Chúng ta có thể giảm bớt 1 phép so sánh nếu chúng ta thêm vào cuối mảng một phần tử cảm canh (sentinel/stand by) có giá trị bằng X để nhận diện ra sự hết mảng khi duyệt mảng, khi đó thuật toán này được cải tiến lại như sau:

```
B1: k = 1
B2: M[N+1] = X //Phần tử cắm canh
B3: IF M[k] ≠ X
    B3.1: k++
    B3.2: Lặp lại B3
B4: IF k < N
    Tìm thấy tại vị trí k
B5: ELSE //k = N song đó chỉ là phần tử cắm canh
    Không tìm thấy phần tử có giá trị X
B6: Kết thúc
```

Hàm LinearSearch được viết lại thành hàm LinearSearch1 như sau:

```
int LinearSearch1 (T M[], int N, T X)
{
    int k = 0;
    M[N] = X;
    while (M[k] != X)
        k++;
    if (k < N)
        return (k);
    return (-1);
}
```

**f. Phân tích thuật toán cải tiến:**

- Trường hợp tốt nhất khi phần tử đầu tiên của mảng có giá trị bằng X:  
Số phép gán:  $G_{min} = 2$   
Số phép so sánh:  $S_{min} = 1 + 1 = 2$
- Trường hợp xấu nhất khi không tìm thấy phần tử nào có giá trị bằng X:  
Số phép gán:  $G_{max} = 2$   
Số phép so sánh:  $S_{max} = (N+1) + 1 = N + 2$
- Trung bình:  
Số phép gán:  $G_{avg} = 2$   
Số phép so sánh:  $S_{avg} = (2 + N + 2) : 2 = N/2 + 2$
- Như vậy, nếu thời gian thực hiện phép gán không đáng kể thì thuật toán cải tiến sẽ chạy nhanh hơn thuật toán nguyên thủy.

**2.2.3. Tìm nhị phân (Binary Search)**

Thuật toán tìm tuyến tính tỏ ra đơn giản và thuận tiện trong trường hợp số phần tử của dãy không lớn lắm. Tuy nhiên, khi số phần tử của dãy khá lớn, chẳng hạn chúng ta tìm kiếm tên một khách hàng trong một danh bạ điện thoại của một thành phố lớn theo thuật toán tìm tuần tự thì quả thực mất rất nhiều thời gian. Trong thực tế, thông thường các phần tử của dãy đã có một thứ tự, do vậy thuật toán tìm nhị phân sau đây sẽ rút ngắn đáng kể thời gian tìm kiếm trên dãy đã có thứ tự. Trong thuật toán này chúng ta giả sử các phần tử trong dãy đã có thứ tự tăng (không giảm dần), tức là các phần tử đứng trước luôn có giá trị nhỏ hơn hoặc bằng (không lớn hơn) phần tử đứng sau nó. Khi đó, nếu  $X$  nhỏ hơn giá trị phần tử đứng ở giữa dãy ( $M[Mid]$ ) thì  $X$  chỉ có thể tìm

thấy ở nửa đầu của dãy và ngược lại, nếu  $X$  lớn hơn phần tử  $M[Mid]$  thì  $X$  chỉ có thể tìm thấy ở nửa sau của dãy.

**a. Tư tưởng:**

Phạm vi tìm kiếm ban đầu của chúng ta là từ phần tử đầu tiên của dãy ( $First = 1$ ) cho đến phần tử cuối cùng của dãy ( $Last = N$ ).

So sánh giá trị  $X$  với giá trị phần tử đứng ở giữa của dãy  $M$  là  $M[Mid]$ .

Nếu  $X = M[Mid]$ : Tìm thấy

Nếu  $X < M[Mid]$ : Rút ngắn phạm vi tìm kiếm về nửa đầu của dãy  $M$  ( $Last = Mid - 1$ )

Nếu  $X > M[Mid]$ : Rút ngắn phạm vi tìm kiếm về nửa sau của dãy  $M$  ( $First = Mid + 1$ )

Lặp lại quá trình này cho đến khi tìm thấy phần tử có giá trị  $X$  hoặc phạm vi tìm kiếm của chúng ta không còn nữa ( $First > Last$ ).

**b. Thuật toán đệ quy (Recursion Algorithm):**

B1:  $First = 1$

B2:  $Last = N$

B3: IF ( $First > Last$ ) //Hết phạm vi tìm kiếm

    B3.1: Không tìm thấy

    B3.2: Thực hiện Bkt

B4:  $Mid = (First + Last) / 2$

B5: IF ( $X = M[Mid]$ )

    B5.1: Tìm thấy tại vị trí  $Mid$

    B5.2: Thực hiện Bkt

B6: IF ( $X < M[Mid]$ )

    Tìm đệ quy từ  $First$  đến  $Last = Mid - 1$

B7: IF ( $X > M[Mid]$ )

    Tìm đệ quy từ  $First = Mid + 1$  đến  $Last$

Bkt: Kết thúc

**c. Cài đặt thuật toán đệ quy:**

Hàm `BinarySearch` có prototype:

```
int BinarySearch (T M[], int N, T X);
```

Hàm thực hiện việc tìm kiếm phần tử có giá trị  $X$  trong mảng  $M$  có  $N$  phần tử đã có thứ tự tăng. Nếu tìm thấy, hàm trả về một số nguyên có giá trị từ 0 đến  $N-1$  là vị trí tương ứng của phần tử tìm thấy. Trong trường hợp ngược lại, hàm trả về giá trị  $-1$  (không tìm thấy). Hàm `BinarySearch` sử dụng hàm đệ quy `RecBinarySearch` có prototype:

```
int RecBinarySearch(T M[], int First, int Last, T X);
```

Hàm `RecBinarySearch` thực hiện việc tìm kiếm phần tử có giá trị  $X$  trên mảng  $M$  trong phạm vi từ phần tử thứ  $First$  đến phần tử thứ  $Last$ . Nếu tìm thấy, hàm trả về một số nguyên có giá trị từ  $First$  đến  $Last$  là vị trí tương ứng của phần tử tìm thấy. Trong trường hợp ngược lại, hàm trả về giá trị  $-1$  (không tìm thấy). Nội dung của các hàm như sau:

```
int RecBinarySearch (T M[], int First, int Last, T X)
{
    if (First > Last)
        return (-1);
    int Mid = (First + Last)/2;
    if (X == M[Mid])
        return (Mid);
    if (X < M[Mid])
        return(RecBinarySearch(M, First, Mid - 1, X));
    else
        return(RecBinarySearch(M, Mid + 1, Last, X));
}

//=====

int BinarySearch (T M[], int N, T X)
{
    return (RecBinarySearch(M, 0, N - 1, X));
}
```

**d. Phân tích thuật toán đệ quy:**

- Trường hợp tốt nhất khi phần tử ở giữa của mảng có giá trị bằng X:  
Số phép gán:  $G_{min} = 1$   
Số phép so sánh:  $S_{min} = 2$
- Trường hợp xấu nhất khi không tìm thấy phần tử nào có giá trị bằng X:  
Số phép gán:  $G_{max} = \log_2 N + 1$   
Số phép so sánh:  $S_{max} = 3\log_2 N + 1$
- Trung bình:  
Số phép gán:  $G_{avg} = \frac{1}{2} \log_2 N + 1$   
Số phép so sánh:  $S_{avg} = \frac{1}{2}(3\log_2 N + 3)$

**e. Thuật toán không đệ quy (Non-Recursion Algorithm):**

```
B1: First = 1
B2: Last = N
B3: IF (First > Last)
    B3.1: Không tìm thấy
    B3.2: Thực hiện Bkt
B4: Mid = (First + Last)/ 2
B5: IF (X = M[Mid])
    B5.1: Tìm thấy tại vị trí Mid
    B5.2: Thực hiện Bkt
B6: IF (X < M[Mid])
    B6.1: Last = Mid - 1
    B6.2: Lặp lại B3
B7: IF (X > M[Mid])
    B7.1: First = Mid + 1
    B7.2: Lặp lại B3
Bkt: Kết thúc
```

**f. Cài đặt thuật toán không đệ quy:**

Hàm NRecBinarySearch có prototype: `int NRecBinarySearch (T M[], int N, T X);`

Hàm thực hiện việc tìm kiếm phần tử có giá trị X trong mảng M có N phần tử đã có thứ tự tăng. Nếu tìm thấy, hàm trả về một số nguyên có giá trị từ 0 đến N-1 là vị trí tương ứng của phần tử tìm thấy. Trong trường hợp ngược lại, hàm trả về giá trị -1 (không tìm thấy). Nội dung của hàm NRecBinarySearch như sau:

```
int NRecBinarySearch (T M[], int N, T X)
{
    int First = 0;
    int Last = N - 1;
    while (First <= Last)
    {
        int Mid = (First + Last)/2;
        if (X == M[Mid])
            return(Mid);
        if (X < M[Mid])
            Last = Mid - 1;
        else
            First = Mid + 1;
    }
    return(-1);
}
```

**g. Phân tích thuật toán không đệ quy:**

- Trường hợp tốt nhất khi phần tử ở giữa của mảng có giá trị bằng X:

Số phép gán:  $G_{min} = 3$

Số phép so sánh:  $S_{min} = 2$

- Trường hợp xấu nhất khi không tìm thấy phần tử nào có giá trị bằng X:

Số phép gán:  $G_{max} = 2\log_2 N + 4$

Số phép so sánh:  $S_{max} = 3\log_2 N + 1$

- Trung bình:

Số phép gán:  $G_{avg} = \log_2 N + 3.5$

Số phép so sánh:  $S_{avg} = \frac{1}{2}(3\log_2 N + 3)$

**h. Ví dụ:**

Giả sử ta có dãy M gồm 10 phần tử có khóa như sau ( $N = 10$ ):

1      3      4      5      8      15      17      22      25      30

- Trước tiên ta thực hiện tìm kiếm phần tử có giá trị  $X = 5$  (tìm thấy):

| Lần lặp | First | Last | First > Last | Mid | M[Mid] | X = M[Mid]  | X < M[Mid] | X > M[Mid] |
|---------|-------|------|--------------|-----|--------|-------------|------------|------------|
| Ban đầu | 0     | 9    | False        | 4   | 8      | False       | True       | False      |
| 1       | 0     | 3    | False        | 1   | 3      | False       | False      | True       |
| 2       | 2     | 3    | False        | 2   | 4      | False       | False      | True       |
| 3       | 3     | 3    | False        | 3   | 5      | <b>True</b> |            |            |
|         |       |      |              |     |        |             |            |            |

Kết quả sau 3 lần lặp (đệ quy) thuật toán kết thúc.

- Bây giờ ta thực hiện tìm kiếm phần tử có giá trị  $X = 7$  (không tìm thấy):

| Lần lặp | First | Last | First > Last | Mid | M[Mid] | $X = M[Mid]$ | $X < M[Mid]$ | $X > M[Mid]$ |
|---------|-------|------|--------------|-----|--------|--------------|--------------|--------------|
| Ban đầu | 0     | 9    | False        | 4   | 8      | False        | True         | False        |
| 1       | 0     | 3    | False        | 1   | 3      | False        | False        | True         |
| 2       | 2     | 3    | False        | 2   | 4      | False        | False        | True         |
| 3       | 3     | 3    | False        | 3   | 5      | False        | False        | True         |
| 4       | 4     | 3    | True         |     |        |              |              |              |
|         |       |      |              |     |        |              |              |              |

Kết quả sau 4 lần lặp (đệ quy) thuật toán kết thúc.

☛ **Lưu ý:**

- Thuật toán tìm nhị phân chỉ có thể vận dụng trong trường hợp dãy/mảng đã có thứ tự. Trong trường hợp tổng quát chúng ta chỉ có thể áp dụng thuật toán tìm kiếm tuần tự.
- Các thuật toán đệ quy có thể ngắn gọn song tốn kém bộ nhớ để ghi nhận mã lệnh chương trình (mỗi lần gọi đệ quy) khi chạy chương trình, do vậy có thể làm cho chương trình chạy chậm lại. Trong thực tế, khi viết chương trình nếu có thể chúng ta nên sử dụng thuật toán không đệ quy.

## **2.3. Các giải thuật tìm kiếm ngoại (Tìm kiếm trên tập tin)**

### **2.3.1. Đặt vấn đề**

Giả sử chúng ta có một tập tin  $F$  lưu trữ  $N$  phần tử. Vấn đề đặt ra là có hay không phần tử có giá trị bằng  $X$  được lưu trữ trong tập tin  $F$ ? Nếu có thì phần tử có giá trị bằng  $X$  là phần tử nằm ở vị trí nào trên tập tin  $F$ ?

### **2.3.2. Tìm tuyến tính**

#### **a. Tư tưởng:**

Lần lượt đọc các phần tử từ đầu tập tin  $F$  và so sánh với giá trị  $X$  cho đến khi đọc được phần tử có giá trị  $X$  hoặc đã đọc hết tập tin  $F$  thì kết thúc.

#### **b. Thuật toán:**

```
B1: k = 0
B2: rewind(F)                //Về đầu tập tin F
B3: read(F, a)               //Đọc một phần tử từ tập tin F
B4: k = k + sizeof(T)        //Vị trí phần tử hiện hành (sau phần tử mới đọc)
B5: IF a ≠ X AND !(eof(F))
    Lặp lại B3
B6: IF (a = X)
    Tìm thấy tại vị trí k byte(s) tính từ đầu tập tin
B7: ELSE
    Không tìm thấy phần tử có giá trị X
```

B8: Kết thúc

**c. Cài đặt thuật toán:**

Hàm FLinearSearch có prototype:

```
long FLinearSearch (char * FileName, T X);
```

Hàm thực hiện tìm kiếm phần tử có giá trị X trong tập tin có tên FileName. Nếu tìm thấy, hàm trả về một số nguyên có giá trị từ 0 đến filelength(FileName) là vị trí tương ứng của phần tử tìm thấy so với đầu tập tin (tính bằng byte). Trong trường hợp ngược lại, hoặc có lỗi khi thao tác trên tập tin hàm trả về giá trị -1 (không tìm thấy hoặc lỗi thao tác trên tập tin). Nội dung của hàm như sau:

```
long FLinearSearch (char * FileName, T X)
{
    FILE * Fp;
    Fp = fopen(FileName, "rb");
    if (Fp == NULL)
        return (-1);
    long k = 0;
    T a;
    int SOT = sizeof(T);
    while (!feof(Fp))
    {
        if (fread(&a, SOT, 1, Fp) == 0)
            break;
        k = k + SOT;
        if (a == X)
            break;
    }
    fclose(Fp);
    if (a == X)
        return (k - SOT);
    return (-1);
}
```

**d. Phân tích thuật toán:**

- Trường hợp tốt nhất khi phần tử đầu tiên của tập tin có giá trị bằng X:

Số phép gán:  $G_{min} = 1 + 2 = 3$

Số phép so sánh:  $S_{min} = 2 + 1 = 3$

Số lần đọc tập tin:  $D_{min} = 1$

- Trường hợp xấu nhất khi không tìm thấy phần tử nào có giá trị bằng X:

Số phép gán:  $G_{max} = N + 2$

Số phép so sánh:  $S_{max} = 2N + 1$

Số lần đọc tập tin:  $D_{max} = N$

- Trung bình:

Số phép gán:  $G_{avg} = \frac{1}{2}(N + 5)$

Số phép so sánh:  $S_{avg} = (3 + 2N + 1) : 2 = N + 2$

Số lần đọc tập tin:  $D_{avg} = \frac{1}{2}(N + 1)$



### **2.3.3. Tìm kiếm theo chỉ mục (Index Search)**

Như chúng ta đã biết, mỗi phần tử dữ liệu được lưu trữ trong tập tin dữ liệu F thường có kích thước lớn, điều này cũng làm cho kích thước của tập tin F cũng khá lớn. Vì vậy việc thao tác dữ liệu trực tiếp lên tập tin F sẽ trở nên lâu, chưa kể sự mất an toàn cho dữ liệu trên tập tin. Để giải quyết vấn đề này, đi kèm theo một tập tin dữ liệu thường có thêm các tập tin chỉ mục (Index File) để làm nhiệm vụ điều khiển thứ tự truy xuất dữ liệu trên tập tin theo một khóa chỉ mục (Index key) nào đó. Mỗi phần tử dữ liệu trong tập tin chỉ mục IDX gồm có 2 thành phần: Khóa chỉ mục và Vị trí vật lý của phần tử dữ liệu có khóa chỉ mục tương ứng trên tập tin dữ liệu. Cấu trúc dữ liệu của các phần tử trong tập tin chỉ mục như sau:

```
typedef struct IdxElement
{
    T      IdxKey;
    long   Pos;
} IdxType;
```

Tập tin chỉ mục luôn luôn được sắp xếp theo thứ tự tăng của khóa chỉ mục. Việc tạo tập tin chỉ mục IDX sẽ được nghiên cứu trong Chương 3, trong phần này chúng ta xem như đã có tập tin chỉ mục IDX để thao tác.

#### **a. Tư tưởng:**

Lần lượt đọc các phần tử từ đầu tập tin IDX và so sánh thành phần khóa chỉ mục với giá trị X cho đến khi đọc được phần tử có giá trị khóa chỉ mục lớn hơn hoặc bằng X hoặc đã đọc hết tập tin IDX thì kết thúc. Nếu tìm thấy thì ta đã có vị trí vật lý của phần tử dữ liệu trên tập tin dữ liệu F, khi đó chúng ta có thể truy cập trực tiếp đến vị trí này để đọc dữ liệu của phần tử tìm thấy.

#### **b. Thuật toán:**

```
B1: rewind(IDX)
B2: read(IDX, ai)
B3: IF ai.IdxKey < X AND !(eof(IDX))
    Lặp lại B2
B4: IF ai.IdxKey = X
    Tìm thấy tại vị trí ai.Pos byte(s) tính từ đầu tập tin
B5: ELSE
    Không tìm thấy phần tử có giá trị X
B6: Kết thúc
```

#### **c. Cài đặt thuật toán:**

Hàm IndexSearch có prototype:

```
long IndexSearch (char * IdxFileName, T X);
```

Hàm thực hiện tìm kiếm phần tử có giá trị X dựa trên tập tin chỉ mục có tên IdxFileName. Nếu tìm thấy, hàm trả về một số nguyên có giá trị từ 0 đến filelength(FileName)-1 là vị trí tương ứng của phần tử tìm thấy so với đầu tập tin dữ liệu (tính bằng byte). Trong trường hợp ngược lại, hoặc có lỗi khi thao tác trên tập tin chỉ mục hàm trả về giá trị -1 (không tìm thấy). Nội dung của hàm như sau:

```
long IndexSearch (char * IdxFilename, T X)
{
    FILE * IDXFP;
    IDXFP = fopen(IdxFilename, "rb");
    if (IDXFP == NULL)
        return (-1);
    IdxType ai;
    int SOIE = sizeof(IdxType);
    while (!feof(IDXFP))
    {
        if (fread(&ai, SOIE, 1, IDXFP) == 0)
            break;
        if (ai.IdxKey >= X)
            break;
    }
    fclose(IDXFP);
    if (ai.IdxKey == X)
        return (ai.Pos);
    return (-1);
}
```

**d. Phân tích thuật toán:**

- Trường hợp tốt nhất khi phần tử đầu tiên của tập tin chỉ mục có giá trị khóa chỉ mục lớn hơn hoặc bằng X:

Số phép gán:  $G_{min} = 1$

Số phép so sánh:  $S_{min} = 2 + 1 = 3$

Số lần đọc tập tin:  $D_{min} = 1$

- Trường hợp xấu nhất khi mọi phần tử trong tập tin chỉ mục đều có khóa chỉ mục nhỏ hơn giá trị X:

Số phép gán:  $G_{max} = 1$

Số phép so sánh:  $S_{max} = 2N + 1$

Số lần đọc tập tin:  $D_{max} = N$

- Trung bình:

Số phép gán:  $G_{avg} = 1$

Số phép so sánh:  $S_{avg} = (3 + 2N + 1) : 2 = N + 2$

Số lần đọc tập tin:  $D_{avg} = \frac{1}{2}(N + 1)$

**Câu hỏi và Bài tập**

1. Trình bày tư tưởng của các thuật toán tìm kiếm: Tuyến tính, Nhị phân, Chỉ mục? Các thuật toán này có thể được vận dụng trong các trường hợp nào? Cho ví dụ?
2. Cài đặt lại thuật toán tìm tuyến tính bằng các cách:
  - Sử dụng vòng lặp for,
  - Sử dụng vòng lặp do ... while?

Có nhận xét gì cho mỗi trường hợp?